POLITECHNIKA MORSKA SZCZECIN

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

Architektura systemów komputerowych

Programowanie w asemblerze

Operacje przypisania, operacje arytmetyczne

mgr inż. Janusz Magaj

j.magaj@pm.szczecin.pl

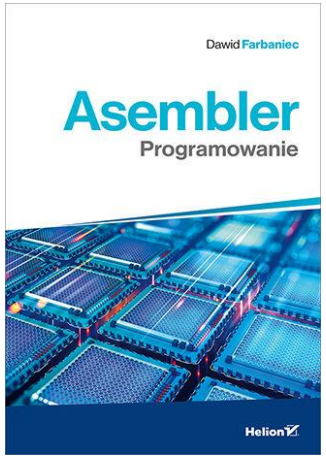
Tematy

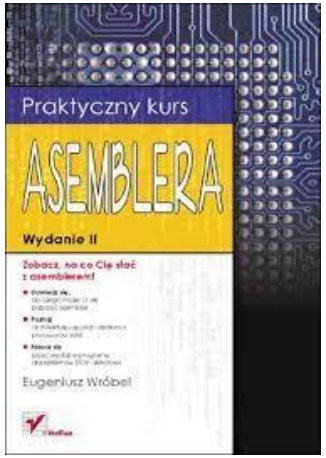
1. Wstęp do architektury komputerów
2. Architektura procesora
3. Reprezentacja i arytmetyka liczb całkowitych w procesorze
4. Reprezentacja i arytmetyka liczb zmiennoprzecinkowych w procesorze
5. **Programowanie w asemblerze**

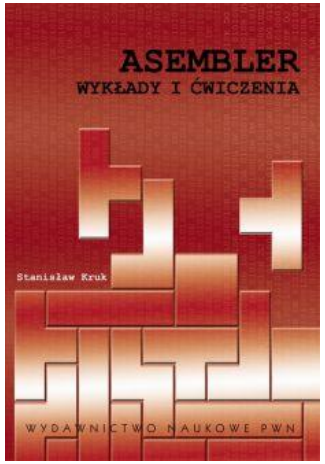
    1. Wstęp - asembler
    2. **Operacje przypisania, operacje arytmetyczne**
    3. Operacje warunkowe
    4. Pętle programowe
    5. Tablice, wskaźniki
    6. Stos, Funkcje, wywołanie funkcji
    7. Operacje zmiennoprzecinkowe
6. Budowa komputera PC

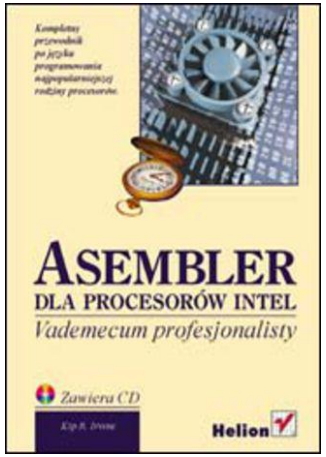
Literatura

- Farbaniec D., Asembler Programowanie, Helion, 2019
- Wróbel E., Praktyczny kurs ASEMBLERA, Helion, 2011
- Kruk S., Asembler Wykłady i ćwiczenia, WN PWN, 2009
- Pirogow V., Asembler. Podręcznik programisty, Helion, 2005
- Irvine K.R., Asembler dla procesorów Intel. Vademecum profesjonalisty, Helion, 2003
- Materiały firmowe – INTEL, AMD, Apple, Qualcomm



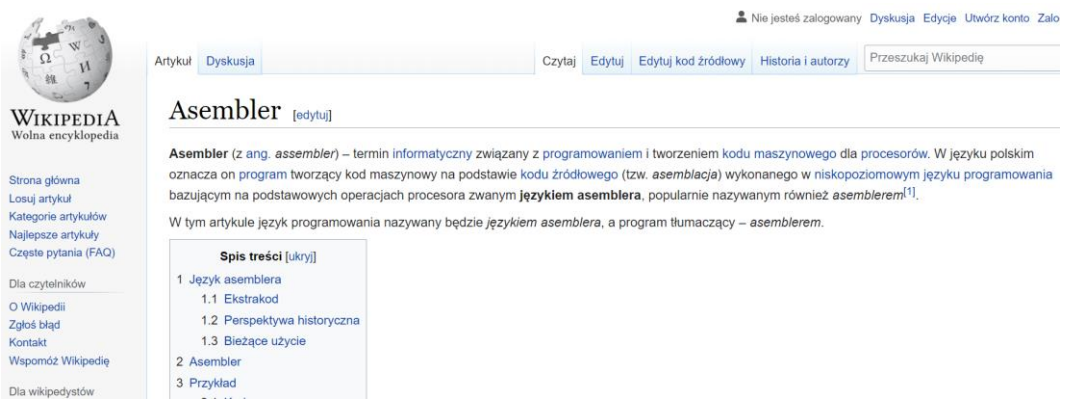




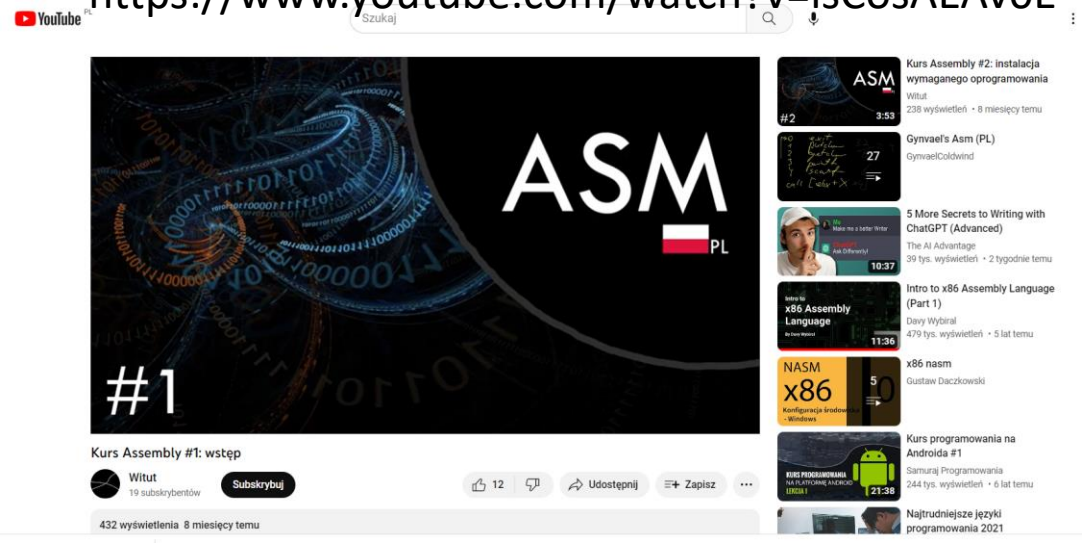


Materiały:

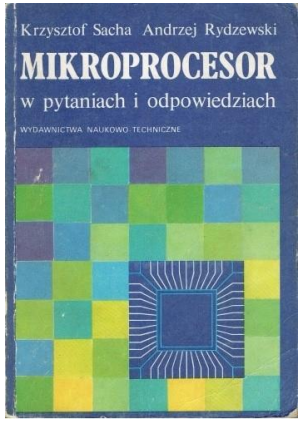
Wiki

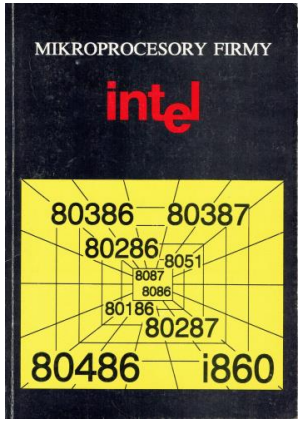


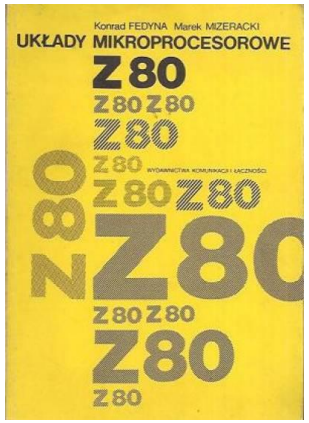
https://www.youtube.com/watch?v=lsCosAEAv0E

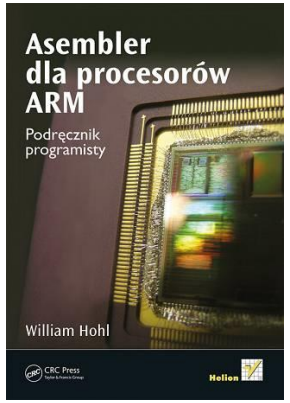


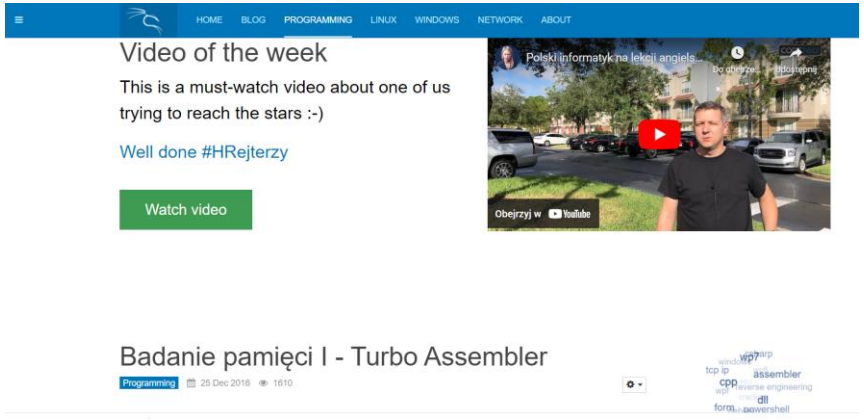
http://blog.prot.com.pl/index.php/programming/20-badanie-pamieci-i-turbo-assembler











Instrukcje w asemblerze

Asembler jest językiem programowania niskiego poziomu, co oznacza, że jednej komendzie asemblera odpowiada dokładnie jeden rozkaz procesora.

```
8b 15 92 80 04 08          mov edx, [koniec]
eb 0a                      jmp koniec
```

- Etykieta koniec oznacza adres **0x08048092** (little endian)

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: mnemonik operandy

- Etykiety (ang. *label*) istnieją jedynie na poziomie kodu asemblerowego.
- Po asemblacji nie pozostaje po nich żaden ślad.
- Służą do nazywania konkretnych adresów w pamięci.
- Znacznie zwiększają czytelność i elastyczność kodu, gdyż operowanie na liczbach określających adresy w pamięci jest skrajnie niewygodne.

  - Długość etykiety od 1 do 31 znaków,
  - Litery, cyfry, znaki specjalne
  - Nie jest rozróżnialna wielkość liter,
  - Spacje są niedozwolone
  - Nie zaczyna się od cyfry

Examples of legal names	Examples of illegal names
COUNTER_1	TWO WORDS
@character	2abc
.TEST	A45.28
DONE?	YOU&ME

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: mnemonik operandy

Aby przydzielić etykietę do adresu, korzystamy z poniższego prostego schematu:

```
etykieta:
```

Etykieta musi znajdować się na początku linii i być niepoprzedzona niczym innym

```
etykieta: movl %ecx, %eax
```

Od tej pory, gdy będziemy chcieli odnieść się do tego adresu (np. gdy zechcemy przeskoczyć w wykonaniu naszego programu bezpośrednio w miejsce z naszą instrukcją ***mov***), możemy napisać po prostu:

```
jmp etykieta
```

zamiast podawać cyferki adresu w pamięci

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: mnemonik operandy

Warto więc pamiętać, że etykieta wskazuje na rozkaz umieszczony w tej samej linii lub następnej niepustej zawierającej instrukcję.

```
        MOV  A,#10
MIEJSCE_SKOKU:  MOV  SBUF,A
        INC  A
        LJMP MIEJSCE_SKOKU

        END
```

```
        MOV  A,#10
MIEJSCE_SKOKU:
        MOV  SBUF,A
        INC  A
        LJMP MIEJSCE_SKOKU
        END
```

Etykieta składająca się z danej nazwy symbolicznej może być zastosowana tylko raz w danym pliku źródłowym.

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: **mnemonik** operandy

- mnemoniki są charakterystyczne dla danego asemblera
- jeden mnemonic może kodować wiele instrukcji maszynowych w zależności od operandów

```
mov al, 200;
```

Mnemonic MOV: [8086,386,X64] Move

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput	
SkylakeX	MOV r,i	1	1	p0156		0.25	
SkylakeX	MOV r8/16,r8/16	1	1	p0156	1	0.25	
SkylakeX	MOV r32/64,r32/64	1	1	p0156	0-1	0.25	may eliminate
SkylakeX	MOV r8l,m	1	2	p23 p0156		0.5	
SkylakeX	MOV r8h,m	1	1	p23		0.5	
SkylakeX	MOV r16,m	1	2	p23 p0156		0.5	
SkylakeX	MOV r32/64,m	1	1	p23	2	0.5	all addressing modes
SkylakeX	MOV m,r	1	2	p237 p4	2	1	
SkylakeX	MOV m,i	1	2	p237 p4		1	

Instrukcje w asemblerze

- jeden **mnemonic** może kodować wiele instrukcji maszynowych w zależności od operandów

Wstęp
Kodowanie 16 bitowe
AAA
AAD
AAM
AAS
ADC
ADD
AND
ARPL
BOUND
BSF
BSR
BSWAP
BT
BTC
BTR
BTS
CALL
CBW
CDQ
CLC

Kodowanie 16 bitowe

Mnemonic: *ADD*

```
00 00                    - ADD [BX+SI],AL
00 01                    - ADD [BX+DI],AL
00 02                    - ADD [BP+SI],AL
00 03                    - ADD [BP+DI],AL
00 04                    - ADD [SI],AL
00 05                    - ADD [DI],AL
00 06 00 00              - ADD [0000],AL
00 07                    - ADD [BX],AL
00 08                    - ADD [BX+SI],CL
00 09                    - ADD [BX+DI],CL
00 0A                    - ADD [BP+SI],CL
00 0B                    - ADD [BP+DI],CL
00 0C                    - ADD [SI],CL
00 0D                    - ADD [DI],CL
00 0E 00 00              - ADD [0000],CL
00 0F                    - ADD [BX],CL
00 10                    - ADD [BX+SI],DL
```

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: **mnemonik** operandy

Producent oferujący mikroprocesor lub mikrokontrolera w dokumentacji dokładnie opisuje poszczególne rozkazy podając **mnemoniki**, efekt działania rozkazu i konkretną wartość liczbową rozkazu dla konkretnego trybu adresowania.

intel® INSTRUCTION SET

26.2.2 How to Read the Instruction Set Pages

The following is an example of the format used for each i486 processor instruction description in this chapter:

CMC—Complement Carry Flag

Opcode	Instruction	Clocks	Description
F5	CMC	2	Complement carry flag

The above table is followed by paragraphs labelled "Operation," "Description," "Flags Affected," "Protected Mode Exceptions," "Real Address Mode Exceptions," and, optionally, "Notes." The following sections explain the notational conventions and abbreviations used in these paragraphs of the instruction descriptions.

26.2.2.1 OPCODE COLUMN

The "Opcode" column gives the complete object code produced for each form of the instruction. When possible, the codes are given as hexadecimal bytes, in the same order in which they appear in memory. Definitions of entries other than hexadecimal bytes are as follows:

/digit: (digit is between 0 and 7) indicates that the ModR/M byte of the instruction uses only the r/m (register or memory) operand. The **reg** field contains the digit that provides an extension to the instruction's opcode.

/r: indicates that the ModR/M byte of the instruction contains both a register operand and an r/m operand.

cb, cw, cd, cp: a 1-byte (cb), 2-byte (cw), 4-byte (cd) or 6-byte (cp) value following the opcode that is used to specify a code offset and possibly a new value for the code segment register.

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: mnemonik **operandy**

- Operandy to argumenty danej instrukcji
- W x86 instrukcjach może ich być od zera do trzech argumentów
- Argumenty oddzielone są przecinkiem (,)
- Jeżeli mamy 2 operandy to pierwszy wskazuje miejsce docelowe a drugi argument źródłowy

```
mov ax, bx
```

Przesłanie do rejestru AX wartości z rejestru BX

stc
inc eax
add eax,ecx
imul eax, ebx,5

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: mnemonik **operandy**

Typy operandów

- **register** - nazwa rejestru
- **memory** odnoszą się do danych w pamięci, adres może być podany bezpośrednio jako część instrukcji lub obliczany na postawie wartości rejestrów
- **immediate** stałe które są częścią instrukcji,
- **implied** operandy domyślne np. 1 w instrukcji zwiększania o jeden.

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: mnemonik **operandy**

- **register**
- **memory**
- **immediate**
- **implied** .

Typy operandów

- **register** - nazwa rejestru

- Rejestry procesora to bardzo szybkie elementy elektroniczne, mogące przechowywać informacje (pamięć wewnętrzna procesora).
- Dostęp do nich i działania na nich są bardzo szybkie.
- Ich nazwy są zakodowane bezpośrednio w kodzie instrukcji maszynowej

Przykładowy rejestr RAX

```
|-------------------------RAX-----------------------------------|
|                            |------------EAX------------------|
|                            |                |-------AX-------|

00000000000000000000000000000000|00000000000000000|00000000|00000000

                                                 |   AH   |  AL    |
```

```
mov ax, bx
```

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: mnemonik **operandy**

Typy operandów

- **register**
- **memory**
- **immediate**
- **implied** .

- **memory** - odnoszą się do danych w pamięci, adres może być podany bezpośrednio jako część instrukcji lub obliczany na postawie wartości rejestrów

```
mov ax, [102h]; Adres końcowy to DS:0 + 102h
```

```
mov ax,[bx + di]
```

```
mov ax,[bx + di + 10]
```

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: mnemonik **operandy**

- **register**
- **memory**
- **immediate**
- **implied** .

Typy operandów

- **immediate** - stałe które są częścią instrukcji

```
mov ax, 1
```

```
mov ax, 010Ch
```

Instrukcje w asemblerze

Instrukcja assemblera ma postać:

etykieta: mnemonik **operandy**

- **register**
- **memory**
- **immediate**
- **implied** .

Typy operandów

- **implied** - operandy domyślne np. 1 w instrukcji zwiększania o jeden

```
inc eax

dec eax
```

Instrukcje w asemblerze

Typy danych

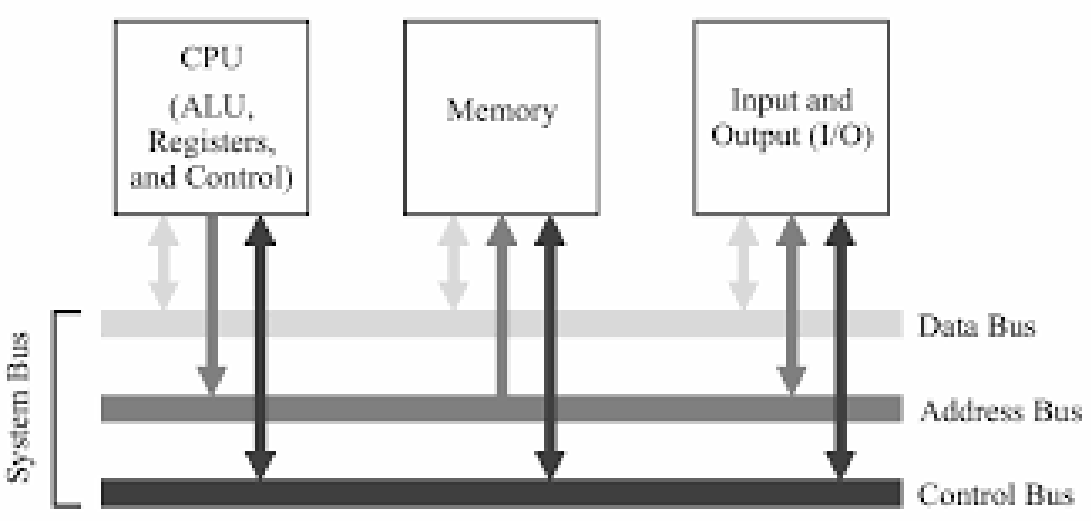
Rozmiary:

Nazwa	Liczba bitów	masm	
byte	8	db	
word	16	dw	
double word (dword)	32	dd	
quadruple word (qword, long)	64	dq	
octuple word (oword)	128	do	

Interpretacja jest zależna od instrukcji:

- liczba całkowita bez znaku
- liczba całkowita ze znakiem
- adres pamięci
- liczba zmiennoprzecinkowa

Adresowanie pamięci

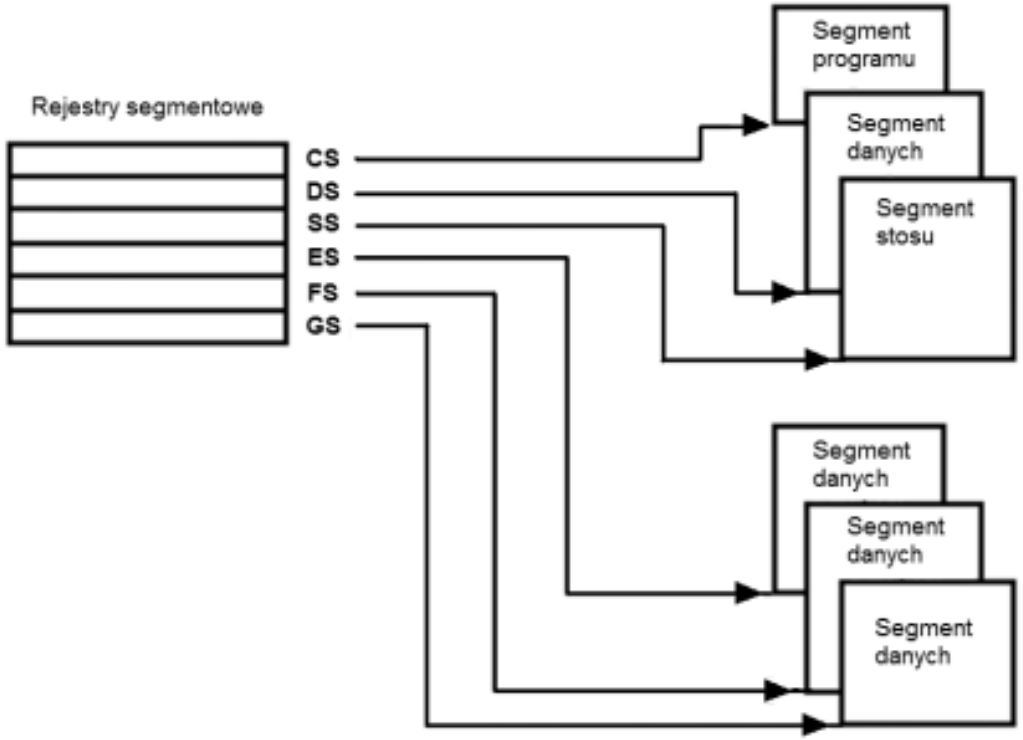


procesor	wielkość magistrali adresowej	max. wielkość pamięci operacyjnej
8088	20 bitów	1 MB
8086	20 bitów	1 MB
80286	24 bity	16 MB
80386	32 bity	4 GB
80486	32 bity	4 GB
80586	32 bity	4 GB
80686	36 bitów	64 GB [2]

- **Segmentowy model pamięci**
- **Płaski model pamięci**

Adresowanie pamięci

- **Segmentowy model pamięci** jest wykorzystywany np. w systemie MS-DOS.
- Filozofia tegoż podziału polega na tym, iż program może mieć w pamięci operacyjnej wydzielone 3 obszary (segmenty): kodu, danych oraz stos.
- Rejestry **CS**, **DS**, **SS** przechowują adresy do tych segmentów.



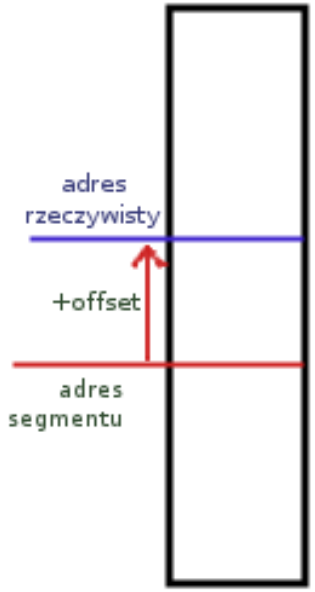
W segmencie kodu przechowywany jest kod naszego programu,
W segmencie danych przechowuje najróżniejsze dane;
Na stosie przechowujemy dane do późniejszego wykorzystania

Adresowanie pamięci

Segmentowy model pamięci

- Odwoływanie się do konkretnego odbywa się przy użyciu dwóch liczb, których rozmiar jest zależny od naszego procesora
- Pierwsza z nich wskazuje na **adres segmentu**, w którym znajduje się komórka; zaś druga liczba jest **przesunięciem**, czyli odległością pomiędzy adresem komórki i adresem początku segmentu, w który się znajduje. Wartość tę nazywa się ***offsetem***.

```
1000:0000
3af8:076b
DS:2ab7
ES:DI
CS:IP
```

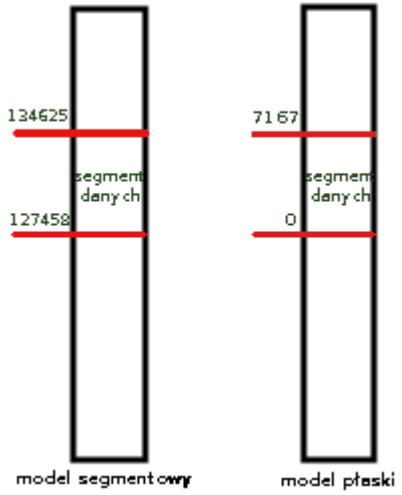


Adresowanie pamięci

Płaski model pamięci (ang. *flat model*)

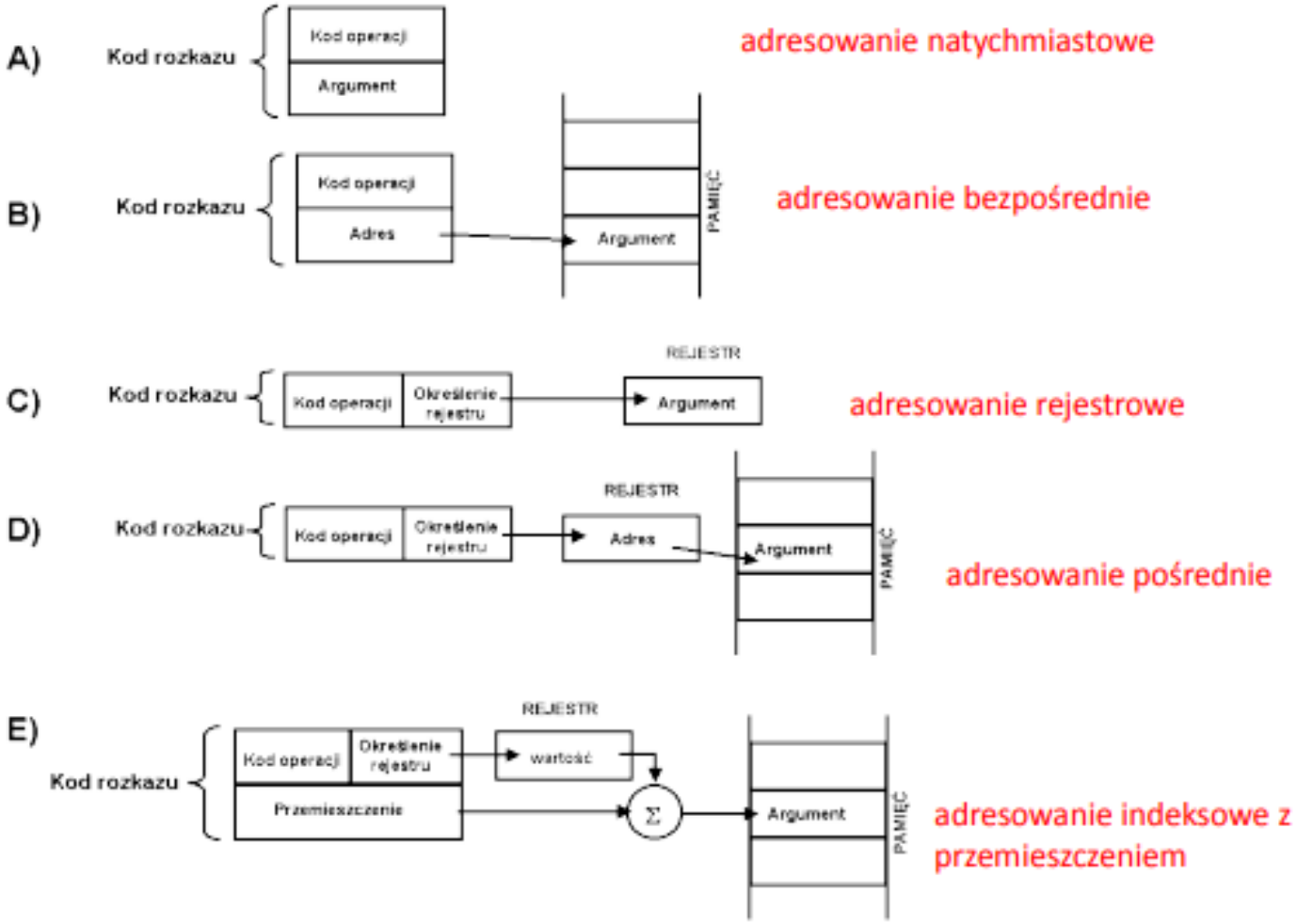
- Jest to sposób organizacji pamięci wykorzystywany przez np. system Windows i systemy z rodziny Unix.
- Jest on odmianą segmentowego modelu pamięci, który jest rozwinięty o ochronę przed zapisem.
- Płaski model pamięci funkcjonuje wyłącznie na procesorach 32-bitowych i tylko w trybie chronionym. Ten model polega na tym, że cała pamięć dostępna dla systemu operacyjnego jest traktowana jako jeden wielki segment. Może on mieć rozmiar maksymalnie 4 GB

Dzięki płaskiemu modelowi pamięci, program jest wykonywany szybciej, gdyż procesor nie musi obliczać adresu komórki jak w adresowaniu ***segment:offset***.



Instrukcje w asemblerze

Tryby adresowania



- **bezpośredni**
- **pośredni przez rejestr**
- **indeksowany**
- **skalowany (ModR/M)**
- **relatywny**

Instrukcje w asemblerze

Tryby adresowania

- **bezpośredni**
- **pośredni przez rejestr**
- **indeksowany**
- **skalowany (ModR/M)**
- **relatywny**

- **bezpośredni**

- podajemy 32 bitową stałą, która określa przesunięcie względem początku pamięci (czyli adresu 0)

MOV AX, [40]

– w rejestrze AX zostanie zapisana zawartość komórki pamięci (segment danych) o adresie 40.

MOV AX, CS:[40]

- rejestrze AX zostanie zapisana zawartość z komórki pamięci (segment PROGRAMU(kodu)) o offsecie 40.

Instrukcje w asemblerze

Tryby adresowania

- bezpośredni
- pośredni przez rejestr
- indeksowany
- skalowany (ModR/M)
- relatywny

- **pośredni przez rejestr**

  - adres znaiduie sie w iednvm z 32/64 bitowych rejestrów

```
mov al, [eax]
mov al, [ebx]
```

80386, 80486, Pentium
- **32 bitowe rejestry**
  EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP, EIP
- **32 bitowy tryb chroniony**: do 4 GB pamięci.

x86-64
- **64 bitowe rejestry**
  RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, RIP
  R8, R9, ..., R15

Instrukcje w asemblerze

Tryby adresowania

- bezpośredni
- pośredni przez rejestr
- indeksowany
- skalowany (ModR/M)
- relatywny

- **indeksowany**

  - efektywny adres wylicza się dodając adres zmiennej i wartość przechowywaną w jednym z rejestrów.

```
mov zmienna[ebx], 5
```

Instrukcje w asemblerze

Tryby adresowania

- **bezpośredni**
- **pośredni przez rejestr**
- **indeksowany**
- **skalowany (ModR/M)**
- **relatywny**

- **skalowany (ModR/M)**

adres = baza + indeks * skala + przesuniecie

gdzie
- baza i indeks znajdują się w rejestrach,
- skala =1,2,4,8,
- przesuniecie jest częścią instrukcji

```
mov [ebx + esi * 4 + 2]
```

Instrukcje w asemblerze

Tryby adresowania

- **bezpośredni**
- **pośredni przez rejestr**
- **indeksowany**
- **skalowany (ModR/M)**
- **relatywny**

- **relatywny**

  - względem aktualnej instrukcji

```
adres = rip + przesuniecie

mov [rip + przesuniecie], 1
```

Rejestry wskaźnikowe

RIP/EIP wskaźnik instrukcji (instruction pointer)
Przechowuje adres następnej instrukcji do wykonania.

Instrukcje w asemblerze

Składnia Intel vs. AT&T

Intel (NASM)

```
neg cx
add eax, 44
shl rsi, cl
mov [rsp-12], edi
mov dword [variable], 0
inc qword [ebp+4*ecx+32]
jmp label
jmp [location]
```

AT&T (GCC, GAS)

```
negw %cx
addl $44, %eax
shlq %cl, %rsi
movq %edi, -12(%rsp)
movl $0, variable
incq 32(%ebp,%ecx,4)
jmp label
jmp *location
```

Instrukcje w asemblerze - podział

Asembler X86 Instrukcje

- przemieszczania danych (przesłań, kopiowania)
- arytmetyczne (binarne, dziesiętne)
- logiczne
- operacje na bitach, bajtach
- skoku
- różne

Instrukcje w asemblerze X86

Rozkazy przesłań, kopiowania

- Wpisz do rejestru konkretną wartość (**MOV**, **LOAD**, **LD**)
- Wpisz do pamięci konkretną wartość
- Ładuj do rejestru roboczego zawartość pamięci lub ładuj do pamięci zawartość rejestru roboczego (**ST**)
- Kopiowanie (przesuwanie) danych z jednego miejsca pamięci w inne miejsce (**MOV**)
- Kopiowanie (przesuwanie) danych z układów (**IN**, **OUT**) wejścia/wyjścia do pamięci lub do rejestrów roboczych
- Zamiana zawartości dwóch rejestrów lub komórek pamięci (**XCHG**)
- Zamiana bitów w rejestrze lub komórce pamięci (**SWAP**)

Instrukcje w asemblerze X86

Rozkazy logiczne

- Rozkazy logiczne operujące na bitach
  – AND, OR, XOR, NOT, ustaw bit, zeruj bit
  (**AND**, **OR**, **XOR**, **ANL**, **ORL**, **XRL**, **CPL**, **SETB**, **CLR**, **SBI**, **CBI**, **TEST**, **NOT**)

- Rozkazy logiczne operujące na bajtach, słowach
  – **AND**, **OR**, **XOR**, **NOT**, przesunięcie w prawo, lewo
  (**RL**, **RRC**, **ROL**, **ROR**, **LSL**, **SHL**, **SRL**)

Instrukcje w asemblerze X86

Rozkazy arytmetyczne

- dodawanie, odejmowanie, (**ADD**, **ADC**, **SUB**, **SBC**)
- mnożenie, dzielenie, (**MUL**, **DIV**)
- zwiększanie lub zmniejszanie o 1, (**INC**, **DEC**)
- porównywanie liczb, (**CMP**, **CP**, **CPI**)
- zamiana (konwersja) liczb, (**DA**, **CBW**, **CWD**)
- rotacje, przesunięcie, (**RL**, **RRC**, **ROL**, **ROR**, **LSL**)
- negacja lub przekształcenie na liczbę w kodzie uzupełnień do dwóch – U2 (**NEG**)

Operacje arytmetyczne mogą być wykonywane:

– na liczbach bez znaku,
– na liczbach ze znakiem,
– na liczbach kodowanych (np. w kodzie BCD, kodzie U2)
– na liczbach stałoprzecinkowych, zmiennoprzecinkowych

Instrukcje w asemblerze X86

Rozkazy – skoki warunkowe i bezwarunkowe

- Skok warunkowy – skok do rozkazu poprzedzonego etykietą jeśli warunek jest prawdziwy
- Skoki warunkowe (ang. jump, branch) od bitu (1 lub 0), bajtu lub słowa (>, <, <=, ≠, =0, ≠0) (**JB**, **JNB**, **JZ**, **JNZ**, **SBIC**, **SBIS**, **BRMI**)
- Skoki od przepełnienia, parzystości
- Skoki bezwarunkowe w konkretne miejsce programu
  – skoki krótkie i skoki dalekie, wywołanie podprogramu, zgłoszenie przerwania (**SJMP**, **LJMP**, **JMP**, **CALL**, **LCALL**, **ICALL**, **GOTO**, **SKIP**)
  – wyjście z podprogramu, wyjście z programu obsługi przerwania (**RET**, **RETURN**, **RETI**)

Instrukcje w asemblerze X86

Rozkazy sterujące pracą procesora

- Przełączanie trybów pracy procesora (**SLEEP**)
- Przełączanie procesorów, np. włączenie FPU(**ESC**)
- Zablokowanie dostępu do magistrali (**LOCK**)
- Zatrzymanie procesora (**HALT**, **HLT**)
- Rozkaz nic nie rób (**NOP**)
- Rozkaz oczekiwania na zdarzenie (**WAIT**)
- Rozkazy operujące na znacznikach w rejestrze stanu – ustawianie i zerowanie bitu (**CLI**, **CLD**, **STD**, **SDI**)
- Zerowanie watchdoga (**WDR**, **CLRWDT**)

Przykłady wybranych instrukcji X86

Intel Assembler 80186 and higher

CodeTable 1/2

© 1996-2003 by Roger Jegerlehner, Switzerland
V 2.3 English. Also available in Spanish

TRANSFER				Flags								
Name	**Comment**	**Code**	**Operation**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
MOV	Move (copy)	MOV Dest,Source	Dest:=Source									
XCHG	Exchange	XCHG Op1,Op2	Op1:=Op2 , Op2:=Op1									
STC	Set Carry	STC	CF:=1									1
CLC	Clear Carry	CLC	CF:=0									0
CMC	Complement Carry	CMC	CF:= ¬CF									±
STD	Set Direction	STD	DF:=1 (string op's downwards)		1							
CLD	Clear Direction	CLD	DF:=0 (string op's upwards)		0							
STI	Set Interrupt	STI	IF:=1			1						
CLI	Clear Interrupt	CLI	IF:=0			0						
PUSH	Push onto stack	PUSH Source	DEC SP, [SP]:=Source									
PUSHF	Push flags	PUSHF	O, D, I, T, S, Z, A, P, C 286+: also NT, IOPL									
PUSHA	Push all general registers	PUSHA	AX, CX, DX, BX, SP, BP, SI, DI									
POP	Pop from stack	POP Dest	Dest:=[SP], INC SP									
POPF	Pop flags	POPF	O, D, I, T, S, Z, A, P, C 286+: also NT, IOPL	±	±	±	±	±	±	±	±	±
POPA	Pop all general registers	POPA	DI, SI, BP, SP, BX, DX, CX, AX									
CBW	Convert byte to word	CBW	AX:=AL (signed)									
CWD	Convert word to double	CWD	DX:AX:=AX (signed)	±				±	±	±	±	±
CWDE	Conv word extended double	CWDE 386	EAX:=AX (signed)									
IN *i*	Input	IN Dest, Port	AL/AX/EAX := byte/word/double of specified port									
OUT *i*	Output	OUT Port, Source	Byte/word/double of specified port := AL/AX/EAX									

i for more information see instruction specifications Flags: ± =affected by this instruction, ?=undefined after this instruction

Przykłady wybranych instrukcji X86

Instrukcja MOV – przesłanie danych

Move (copy)

mov operand docelowy, operand źródłowy

Mnemonic MOV: [8086,386,X64] Move

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput	
SkylakeX	MOV r,i	1	1	p0156		0.25	
SkylakeX	MOV r8/16,r8/16	1	1	p0156	1	0.25	
SkylakeX	MOV r32/64,r32/64	1	1	p0156	0-1	0.25	may eliminate
SkylakeX	MOV r8l,m	1	2	p23 p0156		0.5	
SkylakeX	MOV r8h,m	1	1	p23		0.5	
SkylakeX	MOV r16,m	1	2	p23 p0156		0.5	
SkylakeX	MOV r32/64,m	1	1	p23	2	0.5	all addressing modes
SkylakeX	MOV m,r	1	2	p237 p4	2	1	

Brak wpływu na rejestr **FLAG**

Przykłady wybranych instrukcji X86

Instrukcja MOV – przesłanie danych

mov operand docelowy, operand źródłowy

```
.data
    wynik dword  01h
.code
main proc

        mov eax,07h      ; kopiowanie wartości 7 do rejestru EAX
        mov ecx,eax      ; kopiowanie wartości rejestru EAX do rejestru ECX
        mov wynik,02h    ; kopiowanie wartości 2 do pamięci (zmienna wynik)
        mov ebx, wynik   ; kopiowanie wartości ze zmiennej wynik do rejestru EBX
        mov wynik, eax   ; kopiowanie wartości rejestru do pamięci (zmienna wynik)
        nop;
```

Rejestry

```
EAX = 00000007 EBX = 00000002 ECX = 00000007
EDX = 00CD1005 ESI = 00CD1005 EDI = 00CD1005
EIP = 00CD102C ESP = 00DDF858 EBP = 00DDF864
EFL = 00000246
```

Pamięć 1

Adres: 0x00CD4000 Kolumny: Automatycznie

```
0x00CD4000  07 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0x00CD4010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Przykłady wybranych instrukcji X86

Instrukcja XCHG - wymiana

Exchange

xchg operand I, operand II

Zamiana wartości dwóch operandów

Mnemonic XCHG: [8086,386,X64] Exchange Register/Memory with Register

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	XCHG r,r	3	3	3p0156	2	1
SkylakeX	XCHG r,m	8	8		23	
implicit lock						

Brak wpływu na rejestr **FLAG**

Przykłady wybranych instrukcji X86

Instrukcja XCHG - wymiana

```
9   .data
10      wynik dword  03h
11  .code
12  main proc
13
14          mov eax,00h      ; do rejestru EAX wprowadzamy 0
15          mov ecx,01h      ; do rejestru ECX wprowadzamy 1
16          xchg eax,ecx     ; zamiana wartości rejestrów EAX i ECX
17          xchg ecx, wynik  ; amiana wartości rejestru ECX i zmienej
18          nop;
```

```
9   .data
10      wynik dword  03h
11  .code
12  main proc
13
14          mov eax,00h
15          mov ecx,01h
16          xchg eax,ecx
17          xchg ecx, wynik
```

```
Rejestry
EAX = 00000000 EBX = 006A9000 ECX = 00000001
EDX = 00C61005 ESI = 00C61005 EDI = 00C61005
EIP = 00C6101A ESP = 005BFE00 EBP = 005BFE0C
EFL = 00000246
```

```
Pamięć 1
Adres: &wynik        Kolumny: Auto
0x00C64000  03 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00C64010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
16          xchg eax,ecx
```

```
Rejestry
EAX = 00000001 EBX = 006A9000 ECX = 00000000
EDX = 00C61005 ESI = 00C61005 EDI = 00C61005
EIP = 00C6101B ESP = 005BFE00 EBP = 005BFE0C
EFL = 00000246
```

```
17          xchg ecx, wynik
18          nop;   ≤ 3 ms upłynęło
```

```
Rejestry
EAX = 00000001 EBX = 006A9000 ECX = 00000003
EDX = 00C61005 ESI = 00C61005 EDI = 00C61005
EIP = 00C61021 ESP = 005BFE00 EBP = 005BFE0C
EFL = 00000246
```

```
Pamięć 1
Adres: &wynik        Kolumny: Auto
0x00C64000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x00C64010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Przykłady wybranych instrukcji X86

Instrukcja BSWAP – zmiana kolejności bajtów w operandzie

bswap operand

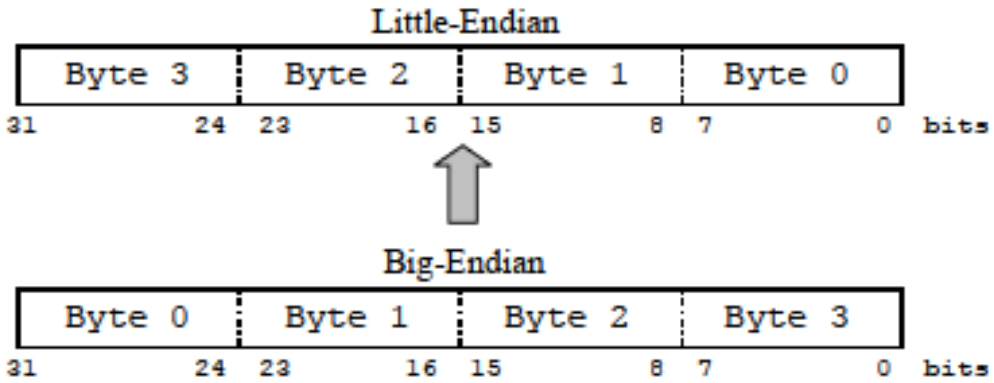
Mnemonic BSWAP: [386,X64,486] Byte Swap

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	BSWAP r32	1	1	p15	1	0.5
SkylakeX	BSWAP r64	2	2	p06 p15	2	1

zamianę kolejności bajtów z **little-endian** na **big-endian** i odwrotnie.(bajty są zapisywane w odwrotnej kolejności)

32/64 - bitowy rejestr ogólnego przeznaczenia.



Przykłady wybranych instrukcji X86

Instrukcja BSWAP – zmiana kolejności bajtów w operandzie

bswap operand

```
14        mov eax,05h          ; do rejestru EAX wprowadzamy 5
15        mov ebx,01020304h    ; do rejestru EBX wprowadzamy daną
16
17        bswap eax     ; zamiana w rejestrze EAX
18        bswap ebx     ; zamiana w rejestrze EBX
19        nop;
```

Rejestry

```
EAX = 00000005 EBX = 01020304 ECX = 00CF1005
  EDX = 00CF1005 ESI = 00CF1005 EDI = 00CF1005
  EIP = 00CF101A ESP = 0115FF68 EBP = 0115FF74
  EFL = 00000246
```

Rejestry

```
EAX = 05000000 EBX = 04030201 ECX = 00CF1005
  EDX = 00CF1005 ESI = 00CF1005 EDI = 00CF1005
  EIP = 00CF101F ESP = 0115FF68 EBP = 0115FF74
  EFL = 00000246
```

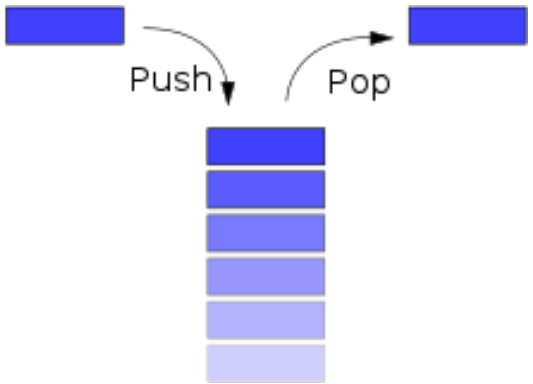
Przykłady wybranych instrukcji X86

Instrukcja PUSH/POP – odłóż na stos/pobierz ze stosu

Mnemonic PUSH: [8086,386,X64] Push Word, Doubleword or Quadword Onto the Stack

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	PUSH r	1	2	p237 p4	3	1
SkylakeX	PUSH i	1	2	p237 p4		1
SkylakeX	PUSH m	2	3	p4 2p237		1
SkylakeX	PUSH stack pointer	2	3	p0156 p237 p4		1



Brak wpływu na rejestr **FLAG**

Mnemonic POP: [8086,386,X64] Pop a Value from the Stack

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	POP r	1	1	p23	2	0.5
SkylakeX	POP stack pointer	3	3	p23 2p0156		3
SkylakeX	POP m	2	3	2p237 p4		1

Przykłady wybranych instrukcji X86

Instrukcja PUSH/POP – odłóż na stos/pobierz ze stosu

```
9   .data
10      wynik dword  03h
11  .code
12  main proc
13
14          mov eax,05h          ; do rejestru EAX wprowadzamy 5
15          mov ebx,02h          ; do rejestru EBX wprowadzamy 2
16          push eax             ; przestanie EAX na stos
17          add eax,ebx          ; dodanie 5+2 wynik w EAX
18          mov wynik, eax       ; do pamięci (wynik) zapamiętuję sumę
19          pop eax              ; odczytanie wartości ze stosu
20
21          nop;
22
```

Rejestry

```
EAX = 00000005 EBX = 00000002 ECX = 00F41005
EDX = 00F41005 ESI = 00F41005 EDI = 00F41005
EIP = 00F4101A ESP = 00AFFC74 EBP = 00AFFC80
EFL = 00000246
```

Pamięć 1

Adres: &wynik

```
0x00F44000  03 00 00 00 00 00
0x00F44010  00 00 00 00 00 00
```

```
10      wynik dword  03h
11  .code
12  main proc
13
14          mov eax,05h
15          mov ebx,02h
16          push eax
17          add eax,ebx
18          mov wynik, eax
19          pop eax
```

Rejestry

```
EAX = 00000007 EBX = 00000002 ECX = 00F41005
EDX = 00F41005 ESI = 00F41005 EDI = 00F41005
EIP = 00F41022 ESP = 00AFFC70 EBP = 00AFFC80
EFL = 00000202
```

Pamięć 1

Adres: &wynik

```
0x00F44000  07 00 00 00 00
0x00F44010  00 00 00 00 00
```

```
14          mov eax,05h
15          mov ebx,02h
16          push eax
17          add eax,ebx
18          mov wynik, eax
19          pop eax
20
21          nop;   ≤ 3 ms upłynęło
```

Rejestry

```
EAX = 00000005 EBX = 00000002 ECX = 00F41005
EDX = 00F41005 ESI = 00F41005 EDI = 00F41005
EIP = 00F41023 ESP = 00AFFC74 EBP = 00AFFC80
EFL = 00000202
```

Przykłady wybranych instrukcji

Instrukcje konwersji

```
CBW    Sign-extend  AL into AX        CBW
CWD    Sign-extend  AX into DX:AX     CWD
CDQ    Sign-extend EAX into EDX:EAX   CDQ
CWDE   Sign-extend  AX into EAX       CWDE
CQO    Sign-extend RAX into RDX:RAX   CQO  (tylko 64bit)
```

Brak wpływu na rejestr **FLAG**

Przykłady wybranych instrukcji

Instrukcja CWD – konwertuje słowo (16 b) na podwójne słowo (32 b)

poprzez rozszerzenie wartości z zachowaniem znaku (ang. Sign-extension)

Mnemonic CWD: [8086] Convert Word to Dword

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	CWD	2	2	p0156	1	1

Brak wpływu na rejestr **FLAG**

```
.data
    wynik dword  03h
.code
main proc
        mov eax,0FFFFFFFFh  ;
        mov ax,1240         ; przestanie 1240 do AX
        neg ax              ; -1240
        cwd                 ; konwersja
        mov wynik, eax      ; do pamięci (wynik)
        nop;   ≤ 2 ms upłynęło
```

Przykłady wybranych instrukcji

Instrukcja CBW – konwertuje bajt (8 b) na słowo (16 b)

poprzez rozszerzenie wartości z zachowaniem znaku (ang. Sign-extension)

Mnemonic CBW: [8086] Convert Byte to Word

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	CBW	1	1	p0156	1	0.5

Brak wpływu na rejestr **FLAG**

```
MOV AL, 60h
CBW
```

AL = 60h = 0110 0000 = +96
AX = 0000 0000 0110 0000 = 0060h = +96
AH = 0000 0000, AL = 0110 0000

```
MOV AL, 80h
CBW
```

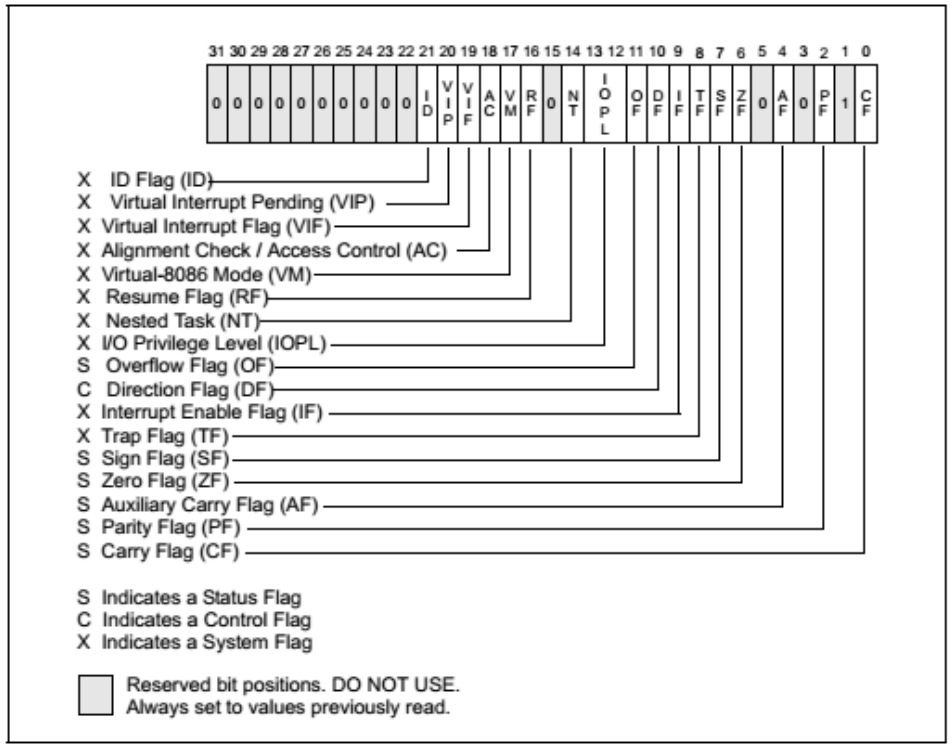
AL = 80h = 1000 0000 = -128
AX = 1111 1111 1000 0000 = FF80h = -128
AH = 1111 1111, AL = 1000 0000

Przykłady wybranych instrukcji

Instrukcja stawienia stanu flagi procesora

Flagi

- CF (carry flag - flaga przeniesienia),
- OF (overflow flag - flaga przepełnienia),
- SF (sign flag - flaga znaku),
- ZF (zero flag - flaga zera),
- IF (interrupt flag - flaga przerwań),
- PF (parity flag - flaga parzystości),
- DF (direction flag - flaga kierunku).



STC	Set Carry	STC
CLC	Clear Carry	CLC
CMC	Complement Carry	CMC
STD	Set Direction	STD
CLD	Clear Direction	CLD
STI	Set Interrupt	STI
CLI	Clear Interrupt	CLI

Przykłady wybranych instrukcji

Instrukcja stawienia stanu flagi przeniesienia CF procesora

STC	Set Carry	STC
CLC	Clear Carry	CLC

```
14        xor eax,eax      ; EAX=0
15        stc              ; ustawienie flagi CF
16        nop              ;
17        clc              ; zerowanie flagi CF
18        nop;
```

Rejestry
```
EAX = 0019FF28 EBX = 005E1000 ECX = 00241005
  EDX = 00241005 ESI = 00241005 EDI = 00241005
  EIP = 00241010 ESP = 0019FED4 EBP = 0019FEE0
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry
```
EAX = 00000000 EBX = 005E1000 ECX = 00241005
  EDX = 00241005 ESI = 00241005 EDI = 00241005
  EIP = 00241012 ESP = 0019FED4 EBP = 0019FEE0
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry
```
EAX = 00000000 EBX = 005E1000 ECX = 00241005
  EDX = 00241005 ESI = 00241005 EDI = 00241005
  EIP = 00241013 ESP = 0019FED4 EBP = 0019FEE0
  EFL = 00000247

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 1
```

Rejestry
```
EAX = 00000000 EBX = 005E1000 ECX = 00241005
  EDX = 00241005 ESI = 00241005 EDI = 00241005
  EIP = 00241014 ESP = 0019FED4 EBP = 0019FEE0
  EFL = 00000247

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 1
```

Rejestry
```
EAX = 00000000 EBX = 005E1000 ECX = 00241005
  EDX = 00241005 ESI = 00241005 EDI = 00241005
  EIP = 00241015 ESP = 0019FED4 EBP = 0019FEE0
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Przykłady wybranych instrukcji

Instrukcja ADD – dodawanie całkowite

add operand docelowy, operand źródłowy

```
.data
    wynik word  03h
.code
main proc

        xor eax,eax
        mov bx,9
        mov ax,5
        add ax,10
        add ax,bx
        add ax,wynik
        add wynik,5
```

Mnemonic ADD: [8086,386,X64] Add

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	ADD r,r/i	1	1	p0156	1	0.25
SkylakeX	ADD r,m	1	2	p0156 p23		0.5
SkylakeX	ADD m,r/i	2	4	2p0156 2p237 p4	5	1

Ustawiany rejestr **FLAG – OF, SF, ZF, CF i PF zależnie od wyniku operacji**

Przykłady wybranych instrukcji

Instrukcja ADD – dodawanie całkowite

```
.data
    wynik word  03h
.code
main proc

        xor eax,eax
        mov bx,9
        mov ax,5
```

```
Rejestry
EAX = 00000005 EBX = 00470009 ECX = 00021005
  EDX = 00021005 ESI = 00021005 EDI = 00021005
  EIP = 0002101A ESP = 0073FD74 EBP = 0073FD80
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Pamięć 1
Adres: &wynik
0x00024000    +3    +0
```

```
add ax,10
```

```
Rejestry
EAX = 0000000F EBX = 00470009 ECX = 00021005
  EDX = 00021005 ESI = 00021005 EDI = 00021005
  EIP = 0002101E ESP = 0073FD74 EBP = 0073FD80
  EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

AX=5+10=15=0Fh

```
add ax,bx
```

```
Rejestry
EAX = 00000018 EBX = 00470009 ECX = 00021005
  EDX = 00021005 ESI = 00021005 EDI = 00021005
  EIP = 00021021 ESP = 0073FD74 EBP = 0073FD80
  EFL = 00000216

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 1 PE = 1 CY = 0
```

AX=15+9=24=18h

```
add ax,wynik
```

```
Rejestry
EAX = 0000001B EBX = 00470009 ECX = 00021005
  EDX = 00021005 ESI = 00021005 EDI = 00021005
  EIP = 00021028 ESP = 0073FD74 EBP = 0073FD80
  EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

AX=24+3=27=1Bh

```
add wynik,5
```

```
Rejestry
EAX = 0000001B EBX = 00470009 ECX = 00021005
  EDX = 00021005 ESI = 00021005 EDI = 00021005
  EIP = 00021028 ESP = 0073FD74 EBP = 0073FD80
  EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

```
Pamięć 1
Adres: &wynik
0x00024000    +8    +0
```

Wynik=3+5=8

Przykłady wybranych instrukcji

Instrukcja ADD – dodawanie całkowite

```
MOV AL, 127        ;AL = +127 = 7Fh
ADD AL, 1          ;AL = -128 = 80h, SF = 1, ZF= 0, OF= 1

MOV AL, 80h        ;AL = 80h = -128
ADD AL, 0FEh       ;AL = 7Eh = +126, SF = 0, ZF = 0, OF = 1
```

```
xor eax,eax
mov al,255
add al,1
```

Rejestry

```
EAX = 000000FF EBX = 01154000 ECX = 00E91005
  EDX = 00E91005 ESI = 00E91005 EDI = 00E91005
  EIP = 00E91014 ESP = 012FF784 EBP = 012FF790
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 00000000 EBX = 01154000 ECX = 00E91005
  EDX = 00E91005 ESI = 00E91005 EDI = 00E91005
  EIP = 00E91016 ESP = 012FF784 EBP = 012FF790
  EFL = 00000257

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 1 PE = 1 CY = 1
```

Przykłady wybranych instrukcji

Instrukcja ADC – dodawanie całkowite z przeniesieniem

adc operand docelowy, operand źródłowy

Mnemonic ADC: [8086,386,X64] Add with Carry

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	ADC r,r/i	1	1	p06	1	1
SkylakeX	ADC r,m	1	2	p06 p23		1
SkylakeX	ADC m,r/i	4	6	3p0156 2p237 p4	5	2

Najczęściej wykonywana gdy dodajemy kilka wartości o rozmiarze bajta, słowa

Ustawiany rejestr **FLAG – OF, SF, ZF, CF i PF zależnie od wyniku operacji**

Przykłady wybranych instrukcji

Instrukcja ADC – dodawanie całkowite z przeniesieniem

Dodawanie dwóch 8-bitowych liczb całkowitych **(FFh + FFh)**, wynik 16-bitowa suma w DL:AL 01h:FEh.

```
xor eax,eax
xor edx,edx
mov al, 0FFh
add al, 0FFh
adc dl, 0
```

Rejestry

```
EAX = 000000FF EBX = 00E62000 ECX = 00C61005
  EDX = 00000000 ESI = 00C61005 EDI = 00C61005
  EIP = 00C61016 ESP = 00B9FC00 EBP = 00B9FC0C
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = 000000FE EBX = 00E62000 ECX = 00C61005
  EDX = 00000000 ESI = 00C61005 EDI = 00C61005
  EIP = 00C61018 ESP = 00B9FC00 EBP = 00B9FC0C
  EFL = 00000293

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 1 PE = 0 CY = 1
```

Rejestry

```
EAX = 000000FE EBX = 00E62000 ECX = 00C61005
  EDX = 00000001 ESI = 00C61005 EDI = 00C61005
  EIP = 00C6101B ESP = 00B9FC00 EBP = 00B9FC0C
  EFL = 00000202

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 0 CY = 0
```

Przykłady wybranych instrukcji

Instrukcja ADC – dodawanie całkowite z przeniesieniem

Dodawanie dwóch 32-bitowych liczb całkowitych **(FFFFFFFFh + FFFFFFFFh),** wynik 64-bitowa suma w EDX:EAX

```
xor eax,eax
xor edx,edx
mov eax, 0FFFFFFFFh
add eax, 0FFFFFFFFh
adc edx, 0
```

Rejestry

```
EAX = FFFFFFFF EBX = 00F85000 ECX = 00B01005
  EDX = 00000000 ESI = 00B01005 EDI = 00B01005
  EIP = 00B01019 ESP = 0116FAB8 EBP = 0116FAC4
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

```
EAX = FFFFFFFE EBX = 00F85000 ECX = 00B01005
  EDX = 00000000 ESI = 00B01005 EDI = 00B01005
  EIP = 00B0101C ESP = 0116FAB8 EBP = 0116FAC4
  EFL = 00000293

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 1 PE = 0 CY = 1
```

Rejestry

```
EAX = FFFFFFFE EBX = 00F85000 ECX = 00B01005
  EDX = 00000001 ESI = 00B01005 EDI = 00B01005
  EIP = 00B0101F ESP = 0116FAB8 EBP = 0116FAC4
  EFL = 00000202

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 0 CY = 0
```

00000001h:FFFFFFFEh

Przykłady wybranych instrukcji

Instrukcja SUB – odejmowanie całkowite

sub operand docelowy, operand źródłowy

Mnemonic SUB: [8086,386,X64] Subtract

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	SUB r,r/i	1	1	p0156	1	0.25
SkylakeX	SUB r,m	1	2	p0156 p23		0.5
SkylakeX	SUB m,r/i	2	4	2p0156 2p237 p4	5	1

```
.data
    wynik word  03h
.code
main proc

        xor eax,eax
        mov bx,4
        mov ax, 15
        sub ax, 2
        sub ax, bx
        sub ax, wynik
        sub wynik,1
```

Ustawiany rejestr **FLAG – OF, SF, ZF, CF i PF zależnie od wyniku operacji**

Przykłady wybranych instrukcji

Instrukcja SUB – odejmowanie całkowite

sub operand docelowy, operand źródłowy

```
.data
    wynik word  03h
.code
main proc

        xor eax,eax
        mov bx,4
        mov ax, 15
        sub ax, 2
        sub ax, bx
        sub ax, wynik
        sub wynik,1
```

```
Rejestry
EAX = 0000000F EBX = 00AF0004 ECX = 008D1005
  EDX = 008D1005 ESI = 008D1005 EDI = 008D1005
  EIP = 008D101A ESP = 00CFFA90 EBP = 00CFFA9C
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Pamięć 1
Adres: 0x008D4000
0x008D4000    +3    +0
```

AX=15, BX=4, Wynik=3

```
Rejestry
EAX = 0000000D EBX = 00AF0004 ECX = 008D1005
  EDX = 008D1005 ESI = 008D1005 EDI = 008D1005
  EIP = 008D101E ESP = 00CFFA90 EBP = 00CFFA9C
  EFL = 00000202

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 0 CY = 0
```

AX=15-2=13=ODh

```
Rejestry
EAX = 00000009 EBX = 00AF0004 ECX = 008D1005
  EDX = 008D1005 ESI = 008D1005 EDI = 008D1005
  EIP = 008D1021 ESP = 00CFFA90 EBP = 00CFFA9C
  EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

AX=13-4=9, Wynik=3

```
Rejestry
EAX = 00000006 EBX = 00AF0004 ECX = 008D1005
  EDX = 008D1005 ESI = 008D1005 EDI = 008D1005
  EIP = 008D1028 ESP = 00CFFA90 EBP = 00CFFA9C
  EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

```
Pamięć 1
Adres: 0x008D4000
0x008D4000    +2    +0
```

AX=9, BX=4, Wynik=3-1=2

Przykłady wybranych instrukcji

Instrukcja MUL – mnożenie bez znaku

- mnożenie wartości w rejestrze AL/AX/EAX/RAX przez wartość z rejestru lub pamięci

mul operand

Mnemonic MUL: [8086,386,X64] Unsigned Multiply

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	MUL r8	1	1	p1	3	1
SkylakeX	MUL r16	4	4	p1 p0156	4	2
SkylakeX	MUL r32	3	3	p1 p0156	4	1
SkylakeX	MUL r64	2	2	p1 p6	3	1
SkylakeX	MUL m8	1	2	p1 p23		1
SkylakeX	MUL m16	4	5	p1 3p0156 p23		2
SkylakeX	MUL m32	3	4	p1 2p0156 p23		2
SkylakeX	MUL m64	2	3	p1 p6 p23		1

```
.data
    wynik word  03h
.code
main proc

        xor eax,eax
        xor ebx,ebx
        mov bx,4
        mov ax,3
        mul bx
        mul wynik
```

Ustawiany rejestr **FLAG – OF, SF, ZF, AF, CF i PF zależnie od wyniku operacji**

Multiplicand	Multiplier	Product
AL	*r/m8*	AX
AX	*r/m16*	DX:AX
EAX	*r/m32*	EDX:EAX

Przykłady wybranych instrukcji

Instrukcja MUL – mnożenie bez znaku

```
.data
    wynik word  03h
.code
main proc

        xor eax,eax
        xor ebx,ebx
        mov bx,4
        mov ax,3
        mul bx
        mul wynik
```

```
Rejestry
EAX = 00000003 EBX = 00000004 ECX = 006B1005
  EDX = 006B1005 ESI = 006B1005 EDI = 006B1005
  EIP = 006B101C ESP = 001EFA28 EBP = 001EFA34
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Pamięć 1
Adres: &wynik
0x006B4000    +3    +0
```

```
Rejestry
EAX = 0000000C EBX = 00000004 ECX = 006B1005
  EDX = 006B0000 ESI = 006B1005 EDI = 006B1005
  EIP = 006B101F ESP = 001EFA28 EBP = 001EFA34
  EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

```
Rejestry
EAX = 00000024 EBX = 00000004 ECX = 006B1005
  EDX = 006B0000 ESI = 006B1005 EDI = 006B1005
  EIP = 006B1026 ESP = 001EFA28 EBP = 001EFA34
  EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 0
```

AX=3, BX=4, Wynik=3

AX=3*4=12=0Ch

AX=12*3=36=24h

AX	*r/m16*	DX:AX

Przykłady wybranych instrukcji

Instrukcja MUL – mnożenie bez znaku

```
xor eax,eax
xor ebx,ebx
xor edx,edx
mov bx,0100h
mov ax,2000h
mul bx
```

```
Rejestry
EAX = 00002000 EBX = 00000100 ECX = 003B1005
  EDX = 00000000 ESI = 003B1005 EDI = 003B1005
  EIP = 003B101E ESP = 006FFE90 EBP = 006FFE9C
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Rejestry
EAX = 00000000 EBX = 00000100 ECX = 003B1005
  EDX = 00000020 ESI = 003B1005 EDI = 003B1005
  EIP = 003B1021 ESP = 006FFE90 EBP = 006FFE9C
  EFL = 00000A07

OV = 1 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 1 CY = 1
```

AX=2000h BX=100h DX=0

AX=2000h*100h=0 i DX=20h

CF = 1, wynik 0020h:0000h in DX:AX

AX	*r/m16*	DX:AX

Przykłady wybranych instrukcji

Instrukcja IMUL – mnożenie ze znakiem

imul operand źródłowy
imul operand docelowy, operand źródłowy
imul operand docelowy, operand źródłowy I, operand źródłowy II

Mnemonic IMUL: [8086,386,X64] Signed Multiply

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	IMUL r8	1	1	p1	3	1
SkylakeX	IMUL r16	4	4	p1 p0156	4	2
SkylakeX	IMUL r32	3	3	p1 p0156	4	1
SkylakeX	IMUL r64	2	2	p1 p6	3	1
SkylakeX	IMUL m8	1	2	p1 p23		1
SkylakeX	IMUL m16	4	5	p1 3p0156 p23		2
SkylakeX	IMUL m32	3	4	p1 2p0156 p23		2
SkylakeX	IMUL m64	2	3	p1 p6 p23		1
SkylakeX	IMUL r,r	1	1	p1	3	1
SkylakeX	IMUL r,m	1	2	p1 p23		1

```
imul eax          ; EDX:EAX = EAX*EAX
imul ebx,ecx,2    ; EBX = ECX*2
imul ebx,ecx      ; EBX = EBX*ECX
imul si,5         ; SI = SI*5
```

Ustawiany rejestr **FLAG – OF, SF, ZF, AF, CF i PF zależnie od wyniku operacji**

Przykłady wybranych instrukcji

Instrukcja IMUL – mnożenie ze znakiem

8-bitowe ze znakiem mnożenie **(48 × 4)**:

```
xor eax,eax
xor ebx,ebx
mov al,48
mov bl,4
imul bl
```

```
Rejestry
EAX = 00000030 EBX = 00000004 ECX = 00591005
  EDX = 00591005 ESI = 00591005 EDI = 00591005
  EIP = 00591018 ESP = 010FFCA4 EBP = 010FFCB0
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Rejestry
EAX = 000000C0 EBX = 00000004 ECX = 00591005
  EDX = 00591005 ESI = 00591005 EDI = 00591005
  EIP = 0059101A ESP = 010FFCA4 EBP = 010FFCB0
  EFL = 00000A87

OV = 1 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 0 PE = 1 CY = 1
```

AX = 00C0h (+192), OF = 1

8-bitowe ze znakiem mnożenie **(-4 × 4)**:

```
xor eax,eax
xor ebx,ebx
mov al,-4
mov bl,4
imul bl
```

Binary value	Sign–magnitude interpretation	Unsigned interpretation
00000000	0	0
00000001	1	1
⋮	⋮	⋮
01111101	125	125
01111110	126	126
01111111	127	127
10000000	−0	128
10000001	−1	129
10000010	−2	130
⋮	⋮	⋮
11111101	−125	253
11111110	−126	254
11111111	−127	255

```
Rejestry
EAX = 000000FC EBX = 00000004 ECX = 005B1005
  EDX = 005B1005 ESI = 005B1005 EDI = 005B1005
  EIP = 005B1018 ESP = 008FF938 EBP = 008FF944
  EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Rejestry
EAX = 0000FFF0 EBX = 00000004 ECX = 005B1005
  EDX = 005B1005 ESI = 005B1005 EDI = 005B1005
  EIP = 005B101A ESP = 008FF938 EBP = 008FF944
  EFL = 00000286

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 0 PE = 1 CY = 0
```

AX = FFF0h, OF = 0

Przykłady wybranych instrukcji

Instrukcja DIV – dzielenie

- dzielenie wartości w rejestrze AL/AX/EAX/RAX przez wartość z rejestru lub pamięci. Wynik w rejestrze A

div operand

Zarówno rejestr jak i A są traktowane jako liczby **bez znaku**.

Dla liczb ze znakiem skorzystaj z **idiv**.

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

```
div ebx
```

Mnemonic DIV: [8086,386,X64] Unsigned Divide

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	DIV r8	10	10	p0 p1 p5 p6	23	6
SkylakeX	DIV r16	10	10	p0 p1 p5 p6	23	6
SkylakeX	DIV r32	10	10	p0 p1 p5 p6	26	6
SkylakeX	DIV r64	36	36	p0 p1 p5 p6	35-88	21-83

Przykłady wybranych instrukcji

Instrukcja DIV – dzielenie

- dzielenie wartości w rejestrze AL/AX/EAX/RAX przez wartość z rejestru lub pamięci. Wynik w rejestrze A

div operand

Zarówno rejestr jak i A są traktowane jako liczby **bez znaku**.

Dla liczb ze znakiem skorzystaj z **idiv**.

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

main.asm

```
.code

main proc

    mov eax,10
    mov ebx,2
    div bl

    nop
main endp
end
```

Rejestry

```
RAX = 000000000000000A RBX = 0000000000000002
```

Rejestry

```
RAX = 0000000000000005 RBX = 0000000000000002
```

Przykłady wybranych instrukcji

Instrukcja DIV – dzielenie

- dzielenie wartości w rejestrze AL/AX/EAX/RAX przez wartość z rejestru lub pamięci. Wynik w rejestrze A

div operand

Zarówno rejestr jak i A są traktowane jako liczby **bez znaku**.

Dla liczb ze znakiem skorzystaj z **idiv**.

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

main.asm

```
.code

main proc

    mov eax,10
    mov ebx,3
    div bl

    nop
main endp
end
```

Rejestry

RAX = 000000000000000A RBX = 0000000000000003

Rejestry

RAX = 0000000000000103 RBX = 0000000000000003

Przykłady wybranych instrukcji

Instrukcja DIV – dzielenie

- dzielenie wartości w rejestrze AL/AX/EAX/RAX przez wartość z rejestru lub pamięci. Wynik w rejestrze A

div operand

Zarówno rejestr jak i A są traktowane jako liczby **bez znaku**.

Dla liczb ze znakiem skorzystaj z **idiv**.

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

main.asm

```
1   .code
2
3   main proc
4       xor rdx,rdx
5       mov rax,1000
6       mov rbx,300
7       div ebx
8
9       nop
10  main endp
11  end
```

Rejestry

```
RAX = 00000000000003E8 RBX = 000000000000012C RCX = 000000D562B1D000 RDX = 0000000000000000
```

Rejestry

```
RAX = 0000000000000003 RBX = 000000000000012C RCX = 000000D562B1D000 RDX = 0000000000000064
```

Przykłady wybranych instrukcji

Instrukcja DIV – dzielenie

- dzielenie wartości w rejestrze AL/AX/EAX/RAX przez wartość z rejestru lub pamięci. Wynik w rejestrze A

div operand

Zarówno rejestr jak i A są traktowane jako liczby **bez znaku**.

Dla liczb ze znakiem skorzystaj z **idiv**.

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

Debug x64 Lokalny d

main.asm

```
1   .data
2      int_wart1 qword  300
3      int_wart2 dw  260
4   .code
5
6   main proc
7       xor rdx,rdx
8       mov rax,int_wart1
9       mov bx,int_wart2
10      div bx
11
12      nop
13  main endp
14  end
```

Rejestry

RAX = 000000000000012C RBX = 0000000000000104 RCX = 00000033E38DD000 RDX = 0000000000000000

Rejestry

RAX = 0000000000000001 RBX = 0000000000000104 RCX = 00000033E38DD000 RDX = 0000000000000028

Przykłady wybranych instrukcji

Instrukcja DIV – dzielenie

- dzielenie wartości w rejestrze AL/AX/EAX/RAX przez wartość z rejestru lub pamięci. Wynik w rejestrze A

div operand

Zarówno rejestr jak i A są traktowane jako liczby **bez znaku**.

Dla liczb ze znakiem skorzystaj z **idiv**.

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

Debug x86 Lokal

main.asm

```
.386
.model flat,stdcall

.data
   int_wart1 dw  300
   int_wart2 dw  260
.code

main proc
    xor edx,edx
    mov ax,int_wart1
    mov bx,int_wart2
    div bx

    nop
main endp
end
```

Rejestry

```
EAX = 0133012C EBX = 011E0104 ECX = 003A1005 EDX = 00000000
```

Rejestry

```
EAX = 01330001 EBX = 011E0104 ECX = 003A1005 EDX = 00000028
```

```
.data
   int_wart1 dw  300
   int_wart2 dw
```

Directive dw: Pseudo-Instruction declaring initialized data. Word, Half Precision (2 bytes, 16

Przykłady wybranych instrukcji

Instrukcja DIV – dzielenie

- dzielenie wartości w rejestrze AL/AX/EAX/RAX przez wartość z rejestru lub pamięci. Wynik w rejestrze A

div operand

Zarówno rejestr jak i A są traktowane jako liczby **bez znaku**.

Dla liczb ze znakiem skorzystaj z **idiv**.

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

Debug x86 Lokal

main.asm

```
.386
.model flat,stdcall

.data
   int_wart1 dw  300
   int_wart2 dw  260
.code

main proc
     xor edx,edx
     mov ax,int_wart1
     mov bx,int_wart2
     div bx

     nop
main endp
end
```

Debug x86 Lokalny

main.asm

```
.386
.model flat,stdcall

.data
   int_wart1 dw  300
   int_wart2 dw  260
.code

main proc
     xor edx,edx
     mov ax,int_wart1
     ;mov bx,int_wart2
     div [int_wart2]

     nop
main endp
end
```

Przykłady wybranych instrukcji

Instrukcja IDIV – dzielenie ze znakiem

Debug x64 Lok

main.asm*

```
.data
    int_wart1 dq  10
    int_wart2 dq  -3
.code

main proc
    xor rdx,rdx
    mov rax,int_wart1
    cdq
    mov rbx,int_wart2
    idiv rbx

    nop
main endp
end
```

10/-3=

Rejestry

RAX = 000000000000000A RBX = FFFFFFFFFFFFFFFD RCX = 00000064C5A31000 RDX = 0000000000000000

Rejestry

RAX = FFFFFFFFFFFFFFFD RBX = FFFFFFFFFFFFFFFD RCX = 000000133F739000 RDX = 0000000000000001

-3 r 1

Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
IDIV r/m8	M	Valid	Valid	Signed divide AX by r/m8, with result stored in: AL := Quotient, AH := Remainder.
IDIV r/m8[1]	M	Valid	N.E.	Signed divide AX by r/m8, with result stored in AL := Quotient, AH := Remainder.
IDIV r/m16	M	Valid	Valid	Signed divide DX:AX by r/m16, with result stored in AX := Quotient, DX := Remainder.
IDIV r/m32	M	Valid	Valid	Signed divide EDX:EAX by r/m32, with result stored in EAX := Quotient, EDX := Remainder.
IDIV r/m64	M	Valid	N.E.	Signed divide RDX:RAX by r/m64, with result stored in RAX := Quotient, RDX := Remainder.

Przykłady wybranych instrukcji

Instrukcja IDIV – dzielenie ze znakiem

-10/-3=

Debug x64 Loka

main.asm

```
.data
   int_wart1 dq  -10
   int_wart2 dq  -3
.code

main proc
    xor rdx,rdx
    mov rax,int_wart1
    cqo
    mov rbx,int_wart2
    idiv rbx

    nop
main endp
end
```

Rejestry

RAX = FFFFFFFFFFFFFFF6 RBX = 0000000000000000 RCX = 000000B8C9EA6000 RDX = 0000000000000000

Rejestry

RAX = FFFFFFFFFFFFFFF6 RBX = 0000000000000000 RCX = 000000B8C9EA6000 RDX = FFFFFFFFFFFFFFFF

Rejestry

RAX = FFFFFFFFFFFFFFF6 RBX = FFFFFFFFFFFFFFFD RCX = 000000B8C9EA6000 RDX = FFFFFFFFFFFFFFFF

Rejestry

RAX = 0000000000000003 RBX = FFFFFFFFFFFFFFFD RCX = 000000B8C9EA6000 RDX = FFFFFFFFFFFFFFFF

3 r -1

Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
IDIV r/m8	M	Valid	Valid	Signed divide AX by r/m8, with result stored in: AL := Quotient, AH := Remainder.
IDIV r/m8[1]	M	Valid	N.E.	Signed divide AX by r/m8, with result stored in AL := Quotient, AH := Remainder.
IDIV r/m16	M	Valid	Valid	Signed divide DX:AX by r/m16, with result stored in AX := Quotient, DX := Remainder.
IDIV r/m32	M	Valid	Valid	Signed divide EDX:EAX by r/m32, with result stored in EAX := Quotient, EDX := Remainder.
IDIV r/m64	M	Valid	N.E.	Signed divide RDX:RAX by r/m64, with result stored in RAX := Quotient, RDX := Remainder.

Przykłady wybranych instrukcji

Instrukcja IDIV – dzielenie ze znakiem

main.asm*

```
.386
.model flat,stdcall

.data
   int_wart1 dd  10
   int_wart2 dd   3
.code

main proc
    xor edx,edx
    mov eax,int_wart1
    mov ebx,int_wart2
    idiv ebx

    nop
main endp
end
```

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

Rejestry

```
EAX = 0000000A EBX = 00000003 ECX = 00301005 EDX = 00000000
```

Rejestry

```
EAX = 00000003 EBX = 00000003 ECX = 00301005 EDX = 00000001
```

3 r 1

Przykłady wybranych instrukcji

Instrukcja IDIV – dzielenie ze znakiem

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

```
4   .data
5      int_wart1 dd  -10
6      int_wart2 dd  -3
7   .code
8
9   main proc
10      xor edx,edx
11      mov eax,int_wart1
12      mov ebx,int_wart2
13      idiv ebx
14
15      nop
16  main endp
17  end
```

```
4   .data
5      int_wart1 dd  -10
6      int_wart2 dd  -3
7   .code
8
9   main proc
10      xor edx,edx
11      mov eax,int_wart1
12      mov ebx,int_wart2
13      idiv bx
14
15      nop
16  main endp
17  end
```

```
div cl  ; AL = (AX div CL), AH = (AX mod CL)
div bx  ; AX = (DX:AX div BX),
        ; DX = (DX:AX mod BX)
div edi ; EAX = (EDX:EAX div EDI),
        ; EDX = (EDX:EAX mod EDI)
div rsi ; RAX = (RDX:RAX div RSI),
        ; RDX = (RDX:RAX mod RSI)
10      xor edx,edx
11      mov eax,int_wart1
12      mov ebx,int_wart2
13      idiv bl
14
15      nop
16  main endp
17  end
```

Rejestry
EAX = AAAAAAAE EBX = FFFFFFFD ECX = 00991005 EDX = 00000000

EAX = AAAAAAAE

???

Rejestry
EAX = FFFFAAAE EBX = FFFFFFFD ECX = 00E81005 EDX = 00000000

EAX = FFFFAAAE

???

Rejestry
EAX = FFFFFF03 EBX = FFFFFFFD ECX = 00481005 EDX = 00000000

EAX = FFFFFF03

3 r -1

Przykłady wybranych instrukcji

Instrukcja IDIV – dzielenie ze znakiem

rozmiar A	rejestry	wynik	reszta
byte	AX	AL	AH
word	DX:AX	AX	DX
double word	EDX:EAX	EAX	EDX

```
4   .data
5      int_wart1 dd  -10
6      int_wart2 dd  -3
7   .code
8
9   main proc
10     xor edx,edx
11     mov eax,int_wart1
12     mov ebx,int_wart2
13     idiv ebx
14
15     nop
16  main endp
17  end
```

```
4   .data
5      int_wart1 dd  -10
6      int_wart2 dd  -3
7   .code
8
9   main proc
10     xor edx,edx
11     mov eax,int_wart1
12     cdq
13     mov ebx,int_wart2
14     idiv bx
15
16     nop
17  main endp
18  end
```

```
cdq
mo
id
no
main e
end
```

Mnemonic CDQ: [8086,386] Convert Dword to Qword

Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	CDQ	1	1	p06	1	1

Rejestry

EAX = FFFFFFF6 EBX = 00D12000 ECX = 00321005 EDX = 00000000

Rejestry

EAX = FFFFFFF6 EBX = 00D12000 ECX = 00321005 EDX = FFFFFFFF

Rejestry

EAX = FFFFFFF6 EBX = FFFFFFFD ECX = 00321005 EDX = FFFFFFFF

Rejestry

EAX = FFFF0003 EBX = FFFFFFFD ECX = 00321005 EDX = FFFFFFFF

Rejestry

EAX = AAAAAAAE EBX = FFFFFFFD ECX = 00991005 EDX = 00000000

EAX = AAAAAAAE

???

EAX = FFFF0003

3 r -1

Przykłady wybranych instrukcji

Przesunięcia i rotacje

SAL Shift arithmetic left	**SAL** Op,Quantity
SAR Shift arithmetic right	**SAR** Op,Quantity
RCL Rotate left through Carry	**RCL** Op,Quantity
RCR Rotate right through Carry	**RCR** Op,Quantity
ROL Rotate left	**ROL** Op,Quantity
ROR Rotate right	**ROR** Op,Quantity

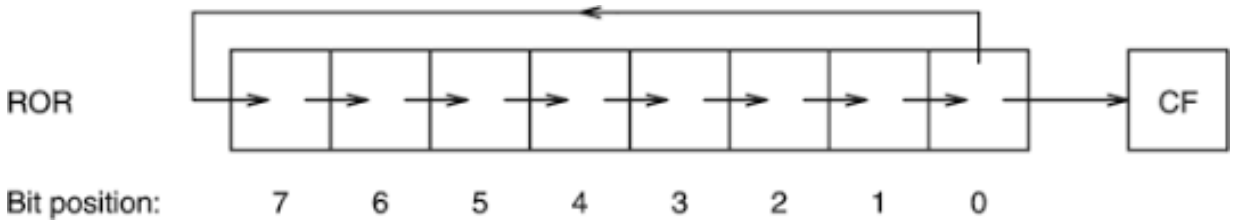
Przykłady wybranych instrukcji

Instrukcja ROR – rotacja w prawo

`Rotate right`

ror operand, przesunięcie

- Przesuwa cyklicznie w prawo o zadaną liczbę bitów.
- Przesuwany jest pierwszy operand, liczba bitów o którą przesuwamy określa drugi operand.
- Najmłodsze bity trafiają do najstarszego bitu za każdym cyklem.
- Jeżeli znak liczby podczas operacji zmieni się (zmieni się pierwszy, najstarszy bit), to znacznik OF jest ustawiany, w przeciwnym wypadku OF jest zerowany.



Mnemonic ROR: [8086,386,X64,186] Rotate Right. Rotate first operand right number of times as given by the second operand.

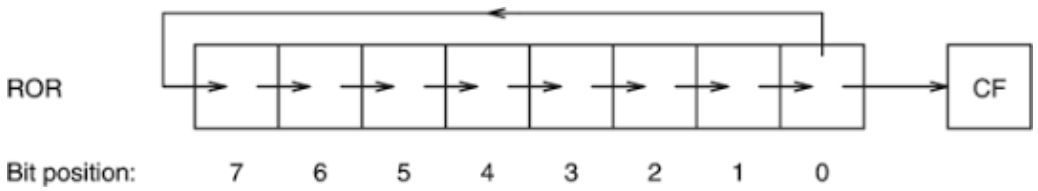
Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput	
SkylakeX	ROR r,1	2	2	2p06	1	1	short form
SkylakeX	ROR r,i	1	1	p06	1	0.5	
SkylakeX	ROR m,i	4	5	2p06 2p237 p4		2	
SkylakeX	ROR r,cl	3	3	3p06	2	2	
SkylakeX	ROR m,cl	5	6	3p06 p23 p4		4	

Przykłady wybranych instrukcji

Instrukcja ROR – rotacja w prawo

ror operand, przesunięcie



	7	6	5	4	3	2	1	0	CF
05h 5	0	0	0	0	0	1	0	1	0
82h 130	1	0	0	0	0	0	1	0	1

```
xor eax,eax
mov ax,5
ror al,1
```

```
EAX = 00000005 EBX = 0084D000 ECX = 00671005 EDX = 00671005
  ESI = 00671005 EDI = 00671005 EIP = 00671016 ESP = 0062F7C0
   EBP = 0062F7CC EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

Rejestry

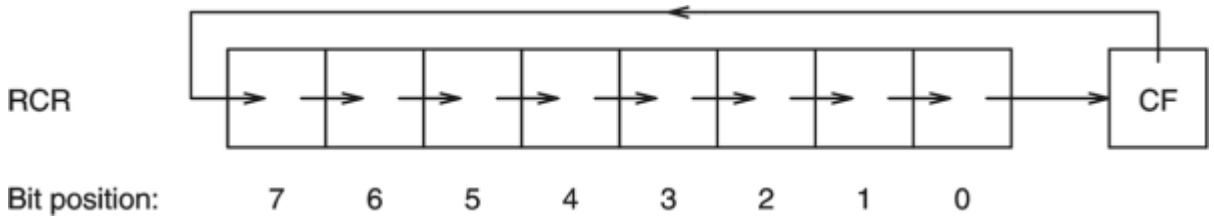
```
EAX = 00000082 EBX = 0084D000 ECX = 00671005 EDX = 00671005
  ESI = 00671005 EDI = 00671005 EIP = 00671018 ESP = 0062F7C0
   EBP = 0062F7CC EFL = 00000A47

OV = 1 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 1
```

Przykłady wybranych instrukcji

Instrukcja RCR – rotacja w prawo z CF

rcr operand , przesunięcie

- Przesuwa cyklicznie w prawo zadaną liczbę bitów, korzystając z znacznika CF.
- Przesuwany jest drugi operand, liczba bitów o którą przesuwamy określa pierwszy operand.
- Najmłodsze bity trafiają do CF, a wartość CF trafia do najstarszego bitu za każdym cyklem.
- Jeżeli znak liczby podczas operacji zmieni się (zmieni się najstarszy bit), to znacznik OF jest ustawiany, w przeciwnym wypadku OF jest zerowany.



Przykłady wybranych instrukcji

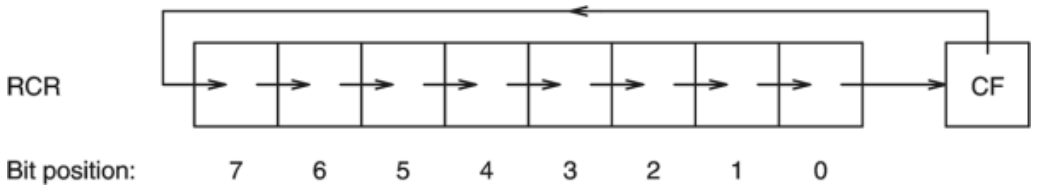
Instrukcja RCR – rotacja w prawo z CF

rcr operand , przesunięcie

```
xor eax,eax
mov ax,5
rcr al,1
```

```
Rejestry
EAX = 00000005 EBX = 00CE3000 ECX = 003E1005 EDX = 003E1005
  ESI = 003E1005 EDI = 003E1005 EIP = 003E1016 ESP = 00EFFC8C
   EBP = 00EFFC98 EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Rejestry
EAX = 00000002 EBX = 00CE3000 ECX = 003E1005 EDX = 003E1005
  ESI = 003E1005 EDI = 003E1005 EIP = 003E1018 ESP = 00EFFC8C
   EBP = 00EFFC98 EFL = 00000247

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 1
```



	7	6	5	4	3	2	1	0	CF
05h 5	0	0	0	0	0	1	0	1	0
02h 2	0	0	0	0	0	0	1	0	1

```
xor eax,eax
mov ax,5
stc
rcr al,1
```

```
Rejestry
EAX = 00000005 EBX = 00E60000 ECX = 00D21005 EDX = 00D21005
  ESI = 00D21005 EDI = 00D21005 EIP = 00D21017 ESP = 00D1FA18
   EBP = 00D1FA24 EFL = 00000247
```

```
Rejestry
EAX = 00000082 EBX = 00E60000 ECX = 00D21005 EDX = 00D21005
  ESI = 00D21005 EDI = 00D21005 EIP = 00D21019 ESP = 00D1FA18
   EBP = 00D1FA24 EFL = 00000A47

OV = 1 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 1
```

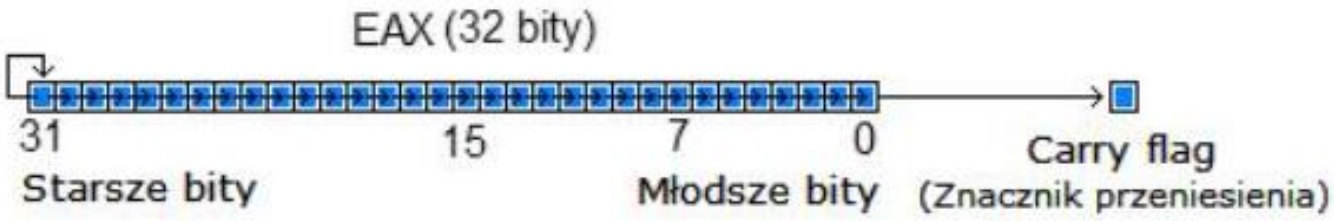
	7	6	5	4	3	2	1	0	CF
05h 5	0	0	0	0	0	1	0	1	1
82h 130	1	0	0	0	0	0	1	0	1

Przykłady wybranych instrukcji

Instrukcja SAR – rotacja w prawo

sar operand , przesunięcie

- Instrukcja SAR przesuwa w prawo bity operandu pierwszego o podaną ilość bitów w operandzie drugim.
- Najmłodszy przesunięty bit, zostaje zachowany w fladze CF, a najstarszy bit zostaje zachowany.
- Instrukcja SHR działa podobnie, różni się tylko tym, że zeruje najstarszy bit (nazywany również bitem znaku).



Mnemonic SAR: [8086,386,X64,186] Shift Arithmetic Right. Shift first operand arithmethic right number of times as given by the second operand.

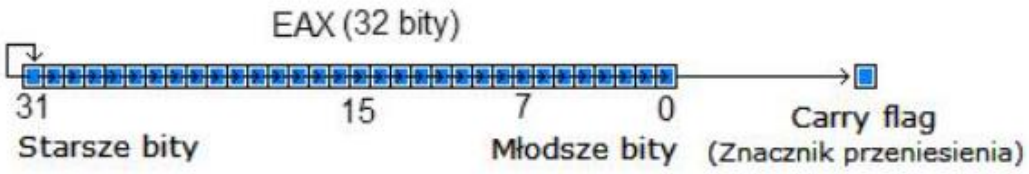
Performance

Architecture	Instruction	µOps Fused	µOps Unfused	µOps Port	Latency	Throughput
SkylakeX	SAR r,i	1	1	p06	1	0.5
SkylakeX	SAR m,i	3	4	2p06 p237 p4		2
SkylakeX	SAR r,cl	3	3	3p06	2	2
SkylakeX	SAR m,cl	5	6	3p06 2p23 p4		4

Przykłady wybranych instrukcji

Instrukcja SAR – rotacja w prawo

sar operand , przesunięcie




```
xor eax,eax
mov ax,5
sar al,1
```

```
Rejestry
EAX = 00000005 EBX = 007CA000 ECX = 00C31005 EDX = 00C31005
ESI = 00C31005 EDI = 00C31005 EIP = 00C31016 ESP = 008FFEA8
EBP = 008FFEB4 EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Rejestry
EAX = 00000002 EBX = 007CA000 ECX = 00C31005 EDX = 00C31005
ESI = 00C31005 EDI = 00C31005 EIP = 00C31018 ESP = 008FFEA8
EBP = 008FFEB4 EFL = 00000203

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 0 AC = 0 PE = 0 CY = 1
```

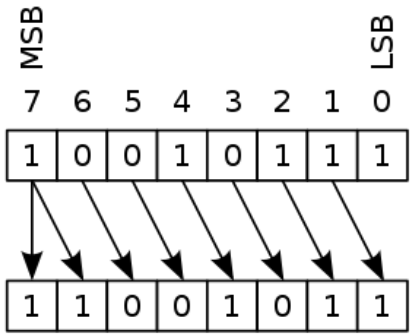
	7	6	5	4	3	2	1	0	CF
5	0	0	0	0	0	1	0	1	0
2	0	0	0	0	0	0	1	0	1

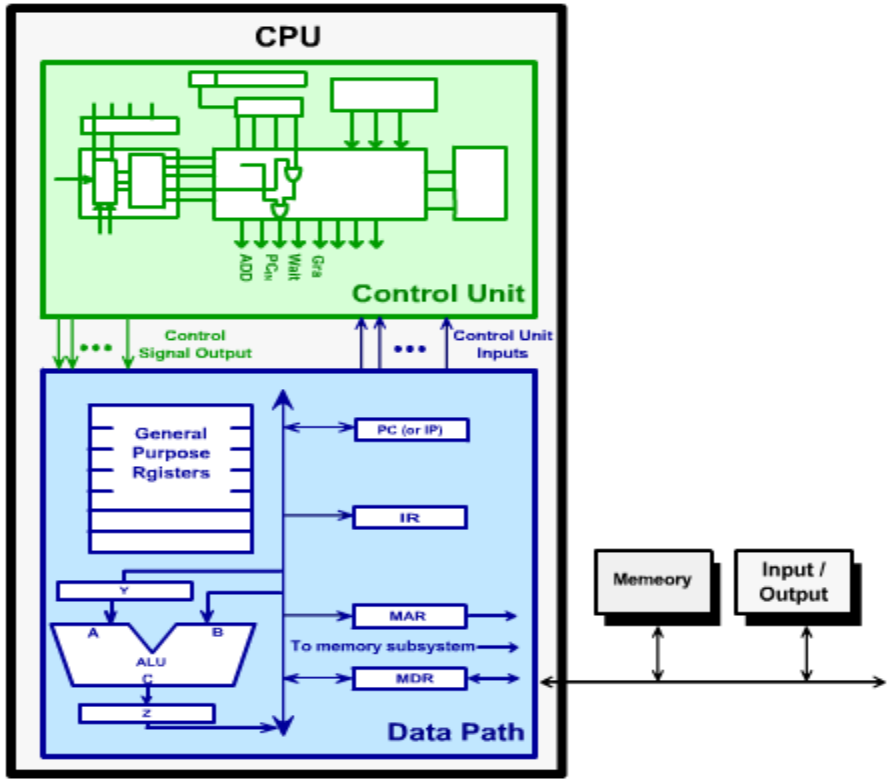
```
xor eax,eax
mov ax,83h
sar al,1
```

```
Rejestry
EAX = 00000083 EBX = 00FDA000 ECX = 003B1005 EDX = 003B1005
ESI = 003B1005 EDI = 003B1005 EIP = 003B1016 ESP = 00DBFF40
EBP = 00DBFF4C EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0 ZR = 1 AC = 0 PE = 1 CY = 0
```

```
Rejestry
EAX = 000000C1 EBX = 00FDA000 ECX = 003B1005 EDX = 003B1005
ESI = 003B1005 EDI = 003B1005 EIP = 003B1018 ESP = 00DBFF40
EBP = 00DBFF4C EFL = 00000283

OV = 0 UP = 0 EI = 1 PL = 1 ZR = 0 AC = 0 PE = 0 CY = 1
```

	7	6	5	4	3	2	1	0	CF
83h 131	1	0	0	0	0	0	1	1	0
C1h 193	1	1	0	0	0	0	0	1	1





Dziękuję za uwagę